

What is my quantum computer good for? Quantum capability learning with physics-aware neural networks

Author: Bohdan Bilash

Affiliation: Quantum Information

Course: Fundamentals of Deep Learning and PyTorch Programming

Date: 2025.12.17

1. Executive Summary

As quantum computers scale, predicting their reliability—specifically which programs (circuits) they can execute successfully—becomes a critical challenge known as "Quantum Capability Learning". Standard deep learning approaches, such as Convolutional Neural Networks (CNNs) and Graph Neural Networks (GNNs), have been applied to this problem but struggle with accuracy and generalization because they treat the quantum device as a "black box," ignoring the underlying physics of quantum noise.

This report analyzes a novel architecture proposed by Hothem et al., the Quantum-Physics-Aware Neural Network (qpa-NN). By embedding physical principles directly into the network structure—specifically the propagability of Pauli errors and the locality of noise—qpa-NNs achieve an approximately 50% reduction in Mean Absolute Error (MAE) compared to state-of-the-art CNNs on both experimental (IBM Quantum) and simulated datasets. Furthermore, the architecture is scalable, demonstrating the first capability model for a 100-qubit device.

This report details the theoretical framework, the specific software implementation (analyzing the provided Python code), the experimental workflow, and the results.

2. Theoretical Framework: The qpa-NN Architecture

The core innovation of the qpa-NN is its hybrid structure. Unlike a standard CNN that attempts to map a circuit diagram directly to a success probability, the qpa-NN splits the problem into two distinct phases: a **learnable error-rate prediction** (Neural Network) and a **fixed physical propagation** (Analytical Function).

2.1 Learning Local Error Rates

The first component is a neural network $\mathcal{N}$ that functions similarly to a Graph Neural Network (GNN). Its goal is not to predict the final fidelity directly, but to predict the rates of specific elementary errors (Hamiltonian H and Stochastic S generators) occurring at each layer of the circuit.

The network assumes that errors are local. An error on qubit q at time t depends primarily on the gates applied to q and its neighbors at that moment. The architecture enforces this by using "filters" that only look at a small window of the circuit around the error location.

2.2 Physics-Based Error Propagation

The second component is a non-trainable function f that takes the predicted error rates and computes the circuit's Probability of Successful Trial (PST) or Process Fidelity (F).

Instead of performing a full density matrix simulation (which is exponentially expensive, $O(4^n)$), the model uses a **first-order Baker-Campbell-Hausdorff (BCH)** approximation. This approximation assumes that weak errors can be summed

linearly and then pushed to the end of the circuit. The formula used for Process Fidelity is:

$$F(c) \approx 1 - \sum_{P \in \mathcal{P}} (s_P + \theta_P^2)$$

Where s_P and θ_P are the aggregate rates of stochastic and coherent errors associated with Pauli term P .

3. Codebase Analysis and Implementation

The provided codebase implements this theory using **TensorFlow** and **Keras**. The implementation is highly modular, separating the circuit encoding, error propagation logic, and the neural network definition.

3.1 Circuit Encoding

Before feeding a circuit into the neural network, it must be encoded into a tensor. The file *encoding.py* handles this. It converts a list of circuit layers into a 3D tensor of shape (num_qubits, depth, num_channels).

Crucially, it also pre-computes the "physics" data needed for the second part of the network: the **Error Propagation Matrix**. The function `create_input_data` calls `create_error_propagation_matrix` (from *newtools.py*) to determine how a Pauli error introduced at layer i transforms as it propagates to the end of the circuit.

Key Code Snippet (*encoding.py*):

```
# Creates the tensor representation of the circuit gates
x_circs[i, :, :] = circuit_to_tensor(c, max_depth, num_qubits,
num_channels, ...)
# Computes how errors propagate (The "Physics" pre-computation)
c_indices, c_signs = create_error_propagation_matrix(c, tracked_error_gens,
stim_dict = stimDict)
# x_indices stores which Pauli term an error becomes at the end of the circuit
# x_signs stores the phase flip (+1 or -1) incurred during propagation
x_indices[i, 0:c.depth, :] = c_indices
x_signs[i, 0:c.depth, :] = c_signs
```

3.2 The Physics-Aware Network Model (*newneuralnets.py*)

The heart of the implementation is in `newneuralnets.py`. The class `CircuitErrorVec` (and its subclass `CircuitErrorVecScreenZErrorsWithMeasurementsBitstrings`) defines the Keras model.

3.2.1 The Localized “Snipper”

The model does not use standard Conv2D layers. Instead, it uses a custom strategy defined in `LocalizedDenseToErrVec`. This layer uses a "snipper" function to extract only the relevant qubits and gates (the local neighborhood) for each specific error generator it is tracking.

Key Code Snippet (*newneuralnets.py*):

```
class LocalizedDenseToErrVec(_keras.layers.Layer):
    def __init__(self, layer_snipper, ...):
        # ...
        # Pre-compute indices to "snip" out relevant parts of the input tensor
        self.layer_encoding_indices_for_error_gen = [
            layer_snipper(error_gen, *layer_snipper_args)
            for error_gen in tracked_error_gens
        ]
    def call(self, inputs):
        # Apply the dense network only to the "snipped" local input
        x = [self.dense[i](_tf.gather(inputs,
self.layer_encoding_indices_for_error_gen[i], axis=-1))
            for i in range(0, self.num_tracked_error_gens)]
        x = _tf.concat(x, axis=-1)
        return x
```

This code enforces the physical principle of locality: the neural network predicting an error on Qubit 1 is mathematically incapable of "seeing" a gate on Qubit 10, preventing it from learning spurious correlations.

3.2.2 The Analytical Output Layer

The *call* method of *CircuitErrorVec* implements the BCH approximation [1]. It takes the error rates predicted by the dense layers (evecs) and combines them using the pre-computed permutation (P) and sign (S) matrices.

Key Code Snippet (*newneuralsnets.py*):

```
def calc_end_of_circ_error(M, P, S, z_mask):  
    """  
    Maps signed error rates (S*M) to an end-of-circuit error generator  
    using permutation matrix P.  
    """  
    signed_M = _tf.math.multiply(S, M)  
    # Filter out Z-errors if necessary (screening)  
    masked_signed_M = _tf.math.multiply(signed_M, z_mask)  
    # Summing error rates that map to the same final Pauli term (BCH approx)  
    final_stochastic_error_rates = calc_masked_err_rates(masked_signed_M, P,  
self.stochastic_mask)  
    final_hamiltonian_error_rates = calc_masked_err_rates(masked_signed_M, P,  
self.hamiltonian_mask)  
    # Fidelity loss = sum(stochastic) + sum(hamiltonian^2)  
    return _tf.reduce_sum(final_stochastic_error_rates) + \  
        _tf.reduce_sum(_tf.square(final_hamiltonian_error_rates))
```

This function is differentiable, allowing the error rates M (the output of the neural network) to be trained via backpropagation using the scalar Fidelity/PST as the label.

3.2 Simulation and Data Generation

The workflow for generating the training data is orchestrated in 1-circuit-generation.ipynb and 2-simulations.ipynb, relying on logic in simfuncs.py.

The system simulates circuits by building a pygsti model or using a custom error model. The `simfuncs.py` file likely contains the logic to sample random circuits and compute their ground truth fidelity using efficient stabilizers (via `stim`) or density matrices (via `pygsti`), although the provided snippet of `simfuncs.py` was not explicitly detailed in the prompt, the notebook `2-simulations.ipynb` clearly drives this process using the `newtools.py` library.

4. Experimental Workflow & Results

The research utilized three distinct data sources, processed via the provided Jupyter notebooks.

4.1 Datasets

1. 5-Qubit Experimental Data (IBM Quantum): Real data from IBM devices (London, Essex, Vigo, etc.). This data is noisy and complex. The model aimed to predict PST (Probability of Successful Trial).
2. 4-Qubit Simulated Data: Generated using `1-circuit-generation.ipynb`. These circuits had controlled noise models (e.g., purely coherent errors) to test specific hypotheses about the model's ability to learn phase errors.
3. 100-Qubit Simulated Data: A massive-scale test to prove scalability. The `CircuitErrorVec` model handles this by simply expanding the graph; the number of parameters in the dense layers remains small because they are local.

4.2 Training Strategy

The training process is documented in `4-training-fidelity.ipynb`. The approach uses standard supervised learning:

- Loss Function: Mean Squared Error (MSE) between predicted and actual Fidelity/PST

- Optimizer: Adam (`_keras.optimizers.Adam`).
- Early Stopping: Used to prevent overfitting.
- Scaling: The targets (Fidelity/PST) are often scaled (e.g., by 10,000) to improve numerical stability during gradient descent.

4.3 Key Results

Superiority over CNNs

On the experimental 5-qubit data, the qpa-NN achieved an average 50.4% reduction in MAE compared to the baseline CNNs. Even when the CNNs were fine-tuned on the same high-fidelity data, the qpa-NNs still outperformed them by ~32%.

Handling Coherent Errors

A major finding (detailed in 7-data-analysis.ipynb and the paper) is that standard CNNs fail to model coherent errors (systematic over/under-rotations). Coherent errors can constructively or destructively interfere (amplitudes add, not probabilities).

- The qpa-NN explicitly squares the sum of Hamiltonian error rates: `_tf.reduce_sum(_tf.square(final_hamiltonian_error_rates))`.
- This allows it to capture the quadratic impact of coherent noise.
- In the 4-qubit simulation with purely coherent errors, the qpa-NN reduced MAE by 52.4% compared to CNNs.

Scalability

The 100-qubit experiment (3-package-circuits.ipynb / 4-training-fidelity.ipynb) demonstrated that the model size does not explode with qubit count.

-
- Figure 3 consists of three panels (a, b, c) showing the comparison of prediction errors for different models. Each panel includes a scatter plot of Prediction error (%) vs Fidelity (%) and marginal histograms for both axes. The top histogram represents Fidelity (%) and the right histogram represents Prediction error (%).
- (a) CNN, MAE = 0.404%; qpa-NN, MAE = 0.176%:** The scatter plot shows prediction errors for the 100-qubit problem. The x-axis (Fidelity) ranges from 90 to 100, and the y-axis (Prediction error) ranges from -4 to 10. The qpa-NN model (blue dots) shows significantly lower prediction errors than the CNN model (orange dots).
 - (b) CNN, MAE = 1.17%; qpa-NN, MAE = 0.72%:** The scatter plot shows prediction errors for the 100-qubit problem with a different fidelity range. The x-axis (Fidelity) ranges from 85 to 100, and the y-axis (Prediction error) ranges from -6 to 10. The qpa-NN model (blue dots) shows lower prediction errors than the CNN model (orange dots).
 - (c) qpa-NN, MAE = 0.0969%:** The scatter plot shows prediction errors for the 100-qubit problem using only the qpa-NN model. The x-axis (Fidelity) ranges from 90 to 100, and the y-axis (Prediction error) ranges from -1 to 2. The qpa-NN model (blue dots) shows very low prediction errors.

5. Discussion and Conclusion

1. **Data Efficiency:** The model learns faster and generalizes better because it is constrained by physics.
2. **Interpretability:** The intermediate layer of the network (the `evecs` in `CircuitErrorVec`) literally represents the error rates of specific gates. This

effectively performs "Quantum Process Tomography" as a side effect of training.

The code analysis confirms that the "Physics-Aware" claim is not just marketing; it is hard-coded into the tensor operations of the Keras layer `LocalizedDenseToErrVec` and the error accumulation logic in `calc_end_of_circ_error`.

Appendix: Key Files Summary

File Name	Function
<code>newneuralnets.py</code>	Core Architecture. Defines <code>CircuitErrorVec</code> , the custom Keras model that integrates the error propagator.
<code>encoding.py</code>	Data Prep. Converts circuit objects into tensors and pre-computes the error propagation (indices/signs).
<code>newtools.py</code>	Physics Engine. Contains the graph theory logic (<code>laplace_from_qubit_graph</code>) and Pauli algebra to track errors.
<code>modeltools.py</code>	GSTi Interface. Bridges the gap between <code>pygsti</code> (standard quantum characterization tool) and the ML pipeline.
<code>4-training-fidelity.ipynb</code>	Training Script. Loads data, instantiates <code>CircuitErrorVec</code> , and runs the Keras <code>fit()</code> loop.

References

[1] Aaronson, Scott, and Daniel Gottesman. "Improved simulation of stabilizer circuits." *Physical Review A—Atomic, Molecular, and Optical Physics* 70.5 (2004): 052328.

[2] Arute, Frank, et al. "Quantum supremacy using a programmable superconducting processor." *Nature* 574.7779 (2019): 505-510.

[3] Campbell, Earl T., Barbara M. Terhal, and Christophe Vuillot. "Roads towards fault-tolerant universal quantum computation." *Nature* 549.7671 (2017): 172-179.